



Practical 4: Singly Linked List: Dynamic Insertion, Deletion, and Traversal

→ Practical Aim

To implement core singly linked list operations and develop the ability to manipulate dynamic sequences through real-world problem scenarios.

Problem 1

A hospital manages a queue of patient tokens. New critical patients must be added to the front, routine patients are added to the end, and occasionally a patient with a priority number must be inserted at a specific position in the queue. Given a sequence of such operations, implement all three insertion types and print the final queue after each operation. Describe how your insertion at a specific position works. What does your solution do if the position given is greater than the current length of the queue? Is that handled explicitly or does it cause an error?

Source Code:

https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/3bf6a94159b8f030dce19878715a4dc3f7a6281d/PRACTICAL%204

OUTPUT

```
"C:\Users\diyat\OneDrive\Docu" x + v
--- HOSPITAL PATIENT QUEUE ---
1. Insert at First
2. Insert at Last
3. Insert at Specific Position
4. Display
5. Exit
Enter your choice: 3
Enter position: 2
Enter patient token: 115
Patient inserted at position 2
Patient Queue:
118 115 104 110

--- HOSPITAL PATIENT QUEUE ---
1. Insert at First
2. Insert at Last
3. Insert at Specific Position
4. Display
5. Exit
Enter your choice: 4
Patient Queue:
118 115 104 110
```

```
"C:\Users\diyat\OneDrive\Doc" X + v
--- HOSPITAL PATIENT QUEUE ---
1. Insert at First
2. Insert at Last
3. Insert at Specific Position
4. Display
5. Exit
Enter your choice: 1
Enter patient token: 104
Patient inserted at first
Patient Queue:
104

--- HOSPITAL PATIENT QUEUE ---
1. Insert at First
2. Insert at Last
3. Insert at Specific Position
4. Display
5. Exit
Enter your choice: 2
Enter patient token: 110
Patient inserted at last
Patient Queue:
104 110

--- HOSPITAL PATIENT QUEUE ---
1. Insert at First
2. Insert at Last
3. Insert at Specific Position
4. Display
5. Exit
Enter your choice: 1
Enter patient token: 118
Patient inserted at first
Patient Queue:
118 104 110
```

The first output shows that patient token 115 was inserted at position 2, and the final queue became 118 115 104 110. The second output shows that patient token 104 was first inserted, then 110 was added at the end, and finally 118 was inserted at the front, giving the final queue 118 104 110.

Conclusion:

The program successfully implements a singly linked list to manage the hospital patient queue. It allows patients to be inserted at the front, end, or a specific position. The final output displays the queue after each operation, and invalid positions are handled without causing an error.

Problem 2

The hospital also needs to remove a patient token from the queue when a patient leaves, print all remaining tokens from last to first for an end-of-day audit, and display the full queue from front to back at any point. Implement deletion by value, reverse printing, and forward traversal on the same queue from Problem 4.1a. Describe how your reverse printing works. Does it use any extra storage, or does it work purely by following the links? Can you think of another way to reverse-print without modifying the original queue?

Source Code

https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/3bf6a94159b8f030dce19878715a4dc3f7a6281d/PRACTICAL%204

OUTPUT

```
"C:\Users\diyat\OneDrive\Do... X + v
Enter number of patients: 8
Enter patient tokens:
110 111 112 113 115 119 120 89

1. Delete patient
2. Forward display
3. Reverse display
4. Exit
Enter choice: 1
Enter token to delete: 89
Queue: 110 111 112 113 115 119 120

1. Delete patient
2. Forward display
3. Reverse display
4. Exit
Enter choice: 2
Queue: 110 111 112 113 115 119 120

1. Delete patient
2. Forward display
3. Reverse display
4. Exit
Enter choice: 3
Reverse Queue: 120 119 115 113 112 111 110

1. Delete patient
2. Forward display
3. Reverse display
4. Exit
Enter choice: 4

Process returned 0 (0x0)   execution time : 50.405 s
Press any key to continue.
```

The output shows that:

- 8 patient tokens were initially added to the queue.
- Token 89 was successfully deleted.
- Forward display shows the remaining tokens: 110 111 112 113 115 119 120
- Reverse display prints them from last to first: 120 119 115 113 112 111 110
- The program exited successfully with return code 0

Conclusion:

The program successfully performs deletion by value, forward traversal, and reverse printing on a singly linked list without changing the original queue structure.

Challenges Faced

Problem 1:

Problem faced : I was confused about what to do when the given position was greater than the list length.

Solution: I added a condition to check the position. If it is invalid, the program displays “Invalid position” and does not insert the node.

Problem 2:

Problem faced:

I found it difficult to handle deletion when the node was the first or the only node.

Solution:

I added a separate condition to update head to the next node. If it was the only node, head becomes NULL.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] Inserting at a position beyond the current length of the list is an invalid operation. Should your solution silently insert at the end, report an error, or do something else entirely? Justify your choice and explain what assumption about the caller your choice makes.

Answer: The solution should report an error instead of silently inserting at the end. This makes the program safer because the given position is assumed to be intentional and valid. If the position is greater than the current length + 1, the program displays “Invalid position” and does not insert the node.

2. [Problem 2] If the node to be deleted is the only node in the list, how many pointers need to change and what must they be set to? How is this case structurally different from deleting a node in the middle?

Answer : If the list has only one node, only one pointer needs to change: head should be set to NULL.

head = NULL ...The deleted node is then freed using free().This is different from deleting a middle node because a middle node has a previous node whose next pointer must also be changed. In the single-node case, there is no previous node.

